Linux Filesystem

Date - 2/10/26 Pranav Tuppada Srivardhan

Introduction to Linux Filesystem

A filesystem is a method of the operating system which is used to store, retrieve, organize and name files on a storage device. A filesystem turns the storage in the storage device into useful meaningful files and folders that the user can work with.
Without a file system storages would just be stored as raw meaning less bytes.

The Linux File System is a structured method of storing and organizing data on a Linux machine. It arranges files in a hierarchical directory format starting from the root directory.
A root directory is the topmost level folder in a computers file system hierarchy from which all other files and folders branch out. Unlike windows which have different directories(like C: or D:) linux follows a single hierarchical tree structure. Root directory is accessed by /.
In Linux, external drives, USBs and additional partitions dont have their own level identity; instead, they are mounted into a specific point within the existing directory tree, becoming part of the same overall structure.

Key Features of a Linux Filesystem

Hierarchical and tree rooted structure - All files and folders on a Linux operating system including user files, system configuration files, software installed, and device files exist within a single root directory /. Other directories are all derived from this root directory, which forms a tree-like hierarchy. Having such a singular point of origin makes a file system deterministic and easier to understand: regardless of the attached hardware or disk partitioning, there will be one topmost starting point.

Cases Sensitive - Linux filesystems are case-sensitive, meaning Files.txt, files.txt, and FILES.txt are treated as three different files. It is different from Windows, which is case-insensitive by default. This distinction matters a lot when writing scripts or working across systems, since a script that works correctly on Linux might fail on Windows if filenames dont match exactly.

Everything is a file - Everything on the system is represented as a file, not just documents and programs, but also hardware devices, running processes, and system information. This abstraction enables the software to interact with system hardware and data streams using standard read/write file operations.

Permission and ownership - Each file and directory in a Linux operating system has an owner, a group, and a specific set of permissions to determine whether it can be read, written, or executed. These permissions apply to the owner, group, and anyone else, and are different from each other. This scheme of permissions is very crucial to the Linux operating system because of its multi-user capability.

Mount points - Linux mounts each file system on a particular directory in the already existing tree. As a result of mounting, all files on that device become a part of the standard directory tree, as if they were always there. This is why, when a user navigates through the file system, he or she will be unable to determine, just from looking at the directory tree, what physical device is used.

Links - Linux supports two kinds of links that let a file be referenced from multiple locations:

- A hard link creates an additional directory entry pointing directly to the same underlying data on disk functionally its almost indistinguishable from the original file.
- A symbolic link (or symlink) is a small special file that simply points to the path of another file. Symlinks can span across different filesystems and even point to files that dont exist, while hard links generally cannot.

Multi user support - Permissions and ownership are intrinsic aspects of the file system structure itself, so Linux is built with multi-user environments right into its heart, allowing many users to work on the same computer or even server concurrently, keeping each of their files and processes separate as per the permissions scheme unless they choose to share.

Directory Structure

Linux follows a standardized layout for its top-level directories, formalized as the Filesystem Hierarchy Standard (FHS).

- /(root) : The top-most level of the entire file system where everything begins.
- /bin : Essential user command binaries (like ls, cp, and mv) needed in single-user mode.
- /sbin : System binaries and administrative commands used by the superuser/root (like reboot).
- /etc : System-wide configuration files and settings for applications (e.g., network and user files).

- /home : Personal folders for regular users where individual files, documents and settings live.
- /root : The home directory specifically for the system administrator.
- /usr : User-space applications, shared programs, libraries and documentation.
- /var : Variable data files that change or grow frequently such as system logs, mail and caches.
- /tmp : Temporary files created by the system or applications, which are usually cleared on reboot.
- /dev : Device files that represent hardware components and peripherals (like hard drive).
- /proc : A virtual file system containing dynamic, real-time information about running processes and the kernel.
- /boot : Static files and bootloader data required to start and load the Linux kernel.
- /lib : Essential shared libraries needed by the core binaries in /bin and /sbin.
- /opt : Optional add-on software packages installed separately from the main system.
- /media /mnt : Temporary mount points used to attach external storage devices like USB drives or file systems.

Basic Commands

To work with the linux filesystem there is a set of basic commands that every user needs to know

Navigation :

- Pwd : Displays the absolute path of the current working directory.
- Ls : Lists the files and folders inside the current directory.
- ls -l : Lists files in a long format, showing sizes, permissions, and owners.
- ls -a : Lists all files, including hidden files starting with a dot.
- cd [directory] : Changes the current working directory to the specified path.
- cd .. : Moves up one level to the parent directory.

Creating and Deleting:

- mkdir [directoryname] : Creates a new empty directory.
- touch [filename] : Creates a new, blank file or updates the timestamp of an existing file.
- rm [filename] : Deletes a specified file.
- rm -r [directoryname] : Recursively deletes a directory and all of its contents.
- rmdir [directoryname] : Deletes a specified directory provided it is empty.

Copying, moving, and renaming

- cp [source] [destination] : Copies a file from one location to another.
- mv [source] [destination] : Moves a file or directory to a new location. Its also used for renaming : moving a file to a new name within the same directory effectively renames it.

Viewing file contents

- cat [file] : Displays the entire contents of a file directly in the terminal. Useful for short files.
- less [file] : Opens a file in a scrollable viewer, better suited for long files than cat.
- head [file] / tail [file] : Displays just the first or last several lines of a file, respectively

Permissions and ownership

- chmod [permissions] [file] : Changes a files permissions.
- chown [user]:[group] [file] : Changes the owner and/or group associated with a file.

Searching

- find [path] -name [pattern] : Searches for files and directories matching a given name pattern, starting from a specified path.
- grep [pattern] [file] : Searches within file contents for lines matching a given text pattern, rather than searching for the files themselves.

Disk and storage information

- df -h : Displays disk space usage across mounted filesystems, in a human-readable format (shows in GB).
- du -h [path] : Shows disk usage for a specific file or directory, also in human-readable units.

Links

- ln [target] [link-name] : Creates a link.

Review Questions

1. Difference between 6 and 9 DOF IMU- The 6 DoF IMU has an accelerometer with a gyroscope (3 axes for both). The 9 DoF IMU contains a magnetometer (another 3 axes), allowing it to have a compass direction since the 6 DoF does not have that and thus it can wander in yaw.

2. The 3 DoF from the magnetic component- The magnetometer measures magnetic field strength along the X, Y, and Z axes, which is used to compute heading relative to magnetic north. These 3 axes are the extra 3 DoF that take a 6-DoF IMU to 9-DoF.

3.Encoder vs IMU for a robotic arm- The encoder should be used for arm joints because it accurately calculates the angle at which the joint turns, which is what is required for arm movement. The IMU calculates the position in relation to gravitational force and the magnetic field, which is not as accurate when it comes to fast and small motions of individual joints.

4.Where would you use analog vs digital camera-

Analog: Older CCTV/security systems, simple low-latency video transmission (like FPV drone feeds) where minimal processing delay matters more than image quality.

Digital: Anywhere image processing, storage, or analysis is needed rovers, modern robotics, computer vision tasks since the output can be directly fed into software for obstacle detection, object recognition, etc.

5.Git reset vs Git revert

- git reset- moves the branch pointer backward, undoing commits (can discard history). Best for local, unpushed changes.
- git revert- creates a new commit that undoes a previous commits changes, keeping history intact. Best for already-pushed/shared changes, since it doesnt rewrite history.

6.Git pull vs Git fetch

- git fetch- downloads changes from the remote but does **not** merge them into your local branch; lets you review first.
- git pull- does fetch + merge in one step, immediately updating your local branch with remote changes.

7. Commands in git config

There are 7 main commands in git config

- git config list : Shows every configuration variable currently set in your environment.
- git config get<key> : Finds and prints the value of a specific key.
- git config set<key><value> : Sets a specific key to a new value.
- git config unset<key> : Completely deletes a configuration key and its value.
- git config rename-section<old name><new name> : Renames an entire group of settings.
- git config remove-section<section name> : Wipes out an entire category of settings at once.
- git config edit : Launches your terminals text editor to manually modify the file.

Review Report 2-

Version Control

Pranav Tuppada Srivardhan

Version Control

Version control (also called source control) is a system that records changes to a file or a set of files over time, so that specific versions can be recalled later. It is used mostly to track changes in the source code with a structured project history. It allows multiple developers to collaborate on the same project without overwriting each others work.
It records the history of the project and every time a change is made , that is recorded as a commit or revision based on the system. Instead of saving a new copy of a file every time you make a change, version control is beneficial in this case.

Version control systems usually offer you the ability to:

- Identify differences between any two versions of code
- Roll back files (or even projects) to an older version
- Determine who made a particular change and when it was made
- Develop several versions of a project at once without one overwriting the other
- Consolidate changes from different developers (or even from different changes from the same developer) into a consolidated version

Why is Version Control Important

Collaboration: It allows multiple people to work on the same document and they all can work without overwriting another persons changes. It provides tools to merge everyones changes together, flagging conflicts only when the same lines of code were changed in incompatible ways.

History and Reverting: Each and every commit is a milestone. If a change brings about an error, that causes the build to break, or turn out to be a mistake in itself, it will always be possible to revert to a previous, functioning version rather than trying to reverse the damage done.

Branching: By using version control, developers are able to make "branches" which refer to separate versions of the codebase that are independent from each other where they can work on developing new features or experiments without altering the main version of the project. Once done the feature can then be merged back into the main branch.

Backup: The version history of a project is saved on a remote server (like GitHub) the project is already backed up. The entire project and all the version histories can still be cloned and retrieved even in cases where the developer loses his laptop or it gets stolen or corrupted.

Revision or Commit: Each revision is always linked with an author and time stamp and always carries a message explaining the change. The log trail allows one to not only see the change made, but also to comprehend the reasoning behind the change, thus allowing a developer to track down the reason for a bug created weeks ago.

Continuous Integration: Version control systems integrate directly with automation tools. Every time you push new code, automated workflows can instantly test, build, and deploy the software, reducing manual work and errors.

Types of Version Control

There are three main types of version control and they are as follows:

1. Local VCS
2. Centralized VCS
3. Distributed VCS

1.Local Version Control

Keeps track of file changes on a single local computer, usually by copying entire files into numbered or dated folders, or by using a simple local database of patches (differences between file versions).

Commands like committing, viewing history, and switching branches happen instantly because the program doesnt have to ping a remote cloud server.

No Collaboration,sharing project states with another person becomes an awkward process of manual file exports.

2.Centralised Version Control

Centralized version control systems introduced a single central server that stores the complete version history of the project.

Programmers will download files to their local computer and do modifications on them, which are subsequently uploaded to this central repository. There exists one single definitive version of the history of the project and that lies in the central server. Each user will have just one working version of the current version in his local machine.

Easier to understand and administer than distributed systems; everyone has visibility into what everyone else is doing, since all commits go through the same server.

The main server is the single point of failure if it crashes or the information contained in it is lost, without proper backup, all the projects history will be lost. Many processes depend on having access to the server through the network.

3.Distributed Version Control

Every contributor has a complete, local copy of the code repository and its entire history. The cloning of a repository by the programmer does not only allow him access to the current files but to the full history of the project. Committing is done on the local machine, and then the commit is pushed or pulled to a remote repository.

Git is an example of distributed version control

Advantage:

- Can work fully offline, since the entire history is available locally
- No single point of failure every clone is effectively a full backup of the project

Disadvantage: Takes time to learn concepts

Important Git Commands

1. git innit- initialises a new repository
2. git clone<repository>- creates a copy of (or downloads) an existing repository
3. git status-shows the present status of the working directory
4. git add <filename>- stages a specific files change, git add . stages all changed files in the current directory
5. git diff- shows the exact line-by-line differences between the working directory and the last commit
6. git commit -m “message”- saves all staged changes as a new commit, along with a descriptive message explaining what was changed
7. git log- displays the commit history, useful for reviewing the evolution of a project
8. git branch- lists all local branches in your repository
9. git switch <branch-name>- switches to a different branch
10. git merge <branch-name>- combines changes from a specified branch into the active one
11. git push origin <branch-name>- uploads your local commits to the remote repository
12. git fetch- downloads history and updates from the remote repository
13. git pull- Fetches and automatically merges remote changes into your current local branch
14. git remote -v- lists the remote repositories connected to your local repo
15. git stash- temporarily shelves uncommitted changes so you can switch branches without losing work
16. git reset- unstages changes or moves the branch pointer backward
17. git revert <commit-id>- creates a new commit that undoes a previous commit's changes, safe for already-pushed history

Bibliography

Brown, P. (2022, February 27). *Classic SysAdmin: The Linux filesystem explained*. The Linux Foundation. https://www.linuxfoundation.org/blog/blog/classic-sysadmin-the-linux-filesystem-explained

Cycle.io. (n.d.). *Filesystem commands*. https://cycle.io/learn/linux-filesystem-commands

GeeksforGeeks. (2026, April 25). *File systems in operating system*. https://www.geeksforgeeks.org/operating-systems/file-systems-in-operating-system/

GeeksforGeeks. (2026, September 10). *Linux file system*. https://www.geeksforgeeks.org/linux-unix/linux-file-system/

Marijan, B. (2026, August 31). *Linux file system: Types, features, limitations*. phoenixNAP. https://phoenixnap.com/kb/linux-file-system